

Problem Statement & Constraints

The Objective: Filter a pool of 100,000 resumes and identify the top 100 best-fit candidates for a **Senior AI Engineer** role at Redrob.

Our architecture is strictly optimized to meet the following non-negotiable constraints:

- **Compute Limits:** CPU-only execution (strictly NO GPU allowed).
- **Memory Bounds:** Peak RAM consumption must remain under 16 GB.
- **Time Budget:** Execution must complete in under 5 minutes wall-clock time.
- **Network Isolation:** 100% offline ranking; no external API calls (e.g. OpenAI, Anthropic, Groq).
- **Intermediate Storage:** Under 5 GB total disk state space.

Multi-Stage Hybrid Pipeline Architecture

To satisfy both speed and recall constraints, we implemented a 3-Stage filtering pipeline:

Stage	Algorithm / Model	Purpose & Execution Details	Output Size
Stage 1: Lexical Retrieval	BM25 Indexing	Rapidly filters the 100,000 candidates down to 2,000 candidates using a highly optimized positive-keyword search query	Top 2,000
Stage 2: Dense Semantic Match	Sentence Transformers (all-MiniLM-L6-v2)	Runs locally on CPU. Encodes the 2,000 retrieved profiles into dense vectors and computes cosine similarity against the	Top 2,000
Stage 3: Behavioral & Penalties	Custom Signals Engine	Applies targeted rules: score bonuses for ideal experience (5-9 years), Github activity, and response rate	Top 2,000

Rank Fusion & Format-Aligned Tie-Breaking

Reciprocal Rank Fusion (RRF):

To combine the semantic similarity scores from Stage 2 and the behavioral alignment scores from Stage 3, we fuse their ranks using RRF (constant $k = 60$) as proposed by Cormack et al. This guarantees robust and balanced integration of lexical, semantic, and resume metadata features.

Precision-Aligned Tie-Breaking:

When writing scores formatted to 6 decimal places (`.6f`), floating point values can experience rounding collisions (e.g. at ranks 87 & 88, both rounding to `0.011188`).

To ensure full compatibility with the schema validator, the pipeline sorts candidates using:

```
key = lambda e: (-float(f"{e['rrf_score']:.6f}"), e['candidate_id'])
```

This guarantees that any identical rounded scores written to the CSV file are perfectly sorted alphabetically by candidate_id, satisfying the challenge's strict non-increasing rank sorting criteria.

Performance & Verification Summary

Metric / Phase	Measured Performance	Hackathon Limit	Compliance Status
Stage 1: Lexical Search (BM25)	48.84 seconds	N/A	Complete
Stage 2: Semantic Encoding	129.45 seconds	N/A	Complete
Stage 3: Behavioral Scoring	0.29 seconds	N/A	Complete
Total Execution Time (100K candidates)	184.18 seconds (3.07 mins)	5.0 minutes (300s)	PASS (38% buffer)
Memory Usage (Peak RAM)	Well below 4 GB	16.0 GB	PASS
Intermediate Disk Usage	Less than 25 MB	5.0 GB	PASS
Validation Check (`validate_submission.py`)	0 Errors (Valid Submission)	No warnings/errors	PASS